

OpenStack on Kubernetes(OpenStack-Helm)환경에서 QEMU 프로세스의 관측 및 선제적 회피 방안

김준석¹

¹숭실대학교 소프트웨어학부 학부생

ssusoftware21@soongsil.ac.kr

Observability and Proactive Avoidance for QEMU Processes in OpenStack-Helm Environments

Joon-Seok Kim¹

¹Dept. of Software Science, Soongsil University

요 약

OpenStack-Helm(OSH)은 OpenStack을 쿠버네티스 파드로 배포하여 관리 편의성을 제공하지만, QEMU 프로세스가 machine(.slice) cgroup 영역에 배치됨에 따라 가상머신(VM)의 자원 사용량과 자원 경합 상태를 관측하지 못하는 한계가 있다. 기존 libvirt 기반 모니터링 역시 실시간 자원 경합 노출에 한계가 있다. 이를 해결하기 위해 본 연구는 libvirt 소켓과 cgroup/procs를 결합하여 자원 사용량과 커널 레벨 경합 지표(PSI)를 추출하는 읽기 전용 수집기(qemu-exporter)를 제안하고, 이를 기반으로 자원 고갈 전 Nova 라이브 마이그레이션을 선제 트리거하는 자동화 파이프라인을 구축하였다. 실험 결과, 0.014%(8 vCPU 기준) 수준의 낮은 오버헤드로 자원 경합을 정밀 관측하고 VM을 성공적으로 회피시킴을 확인하였다.

1. 서론

쿠버네티스의 운영상 이점에 따라 쿠버네티스의 도입률이 높아지고, [1] 이에 맞춰 오픈스택 생태계에서는 OpenStack-Helm(OSH)을 활용해 컴포넌트를 파드(Pod) 형태로 배포, 관리하는 구조가 활성화되고 있다. OSH 환경에서 VM 생성 시 Nova는 libvirt 파드를 거쳐 QEMU 프로세스를 실행하지만, 해당 프로세스는 파드 cgroup(kubepods.slice)이 아닌 호스트 루트의 machine(.slice) cgroup에 배치된다[2]. 이로 인해 쿠버네티스는 VM의 자원 사용을 인식하지 못하며, 기존 libvirt API 기반 모니터링 역시 실시간 자원 경합을 정밀히 포착하지 못한다. 이에 본 연구에서는 소스코드 수정 없이 자원 사용량과 실시간 자원 경합(PSI)을 관측하는 읽기 전용(Read-Only) 수집기를 구현하고, 이를 활용해 노드 자원 고갈 전 Nova 라이브 마이그레이션(Live Migration)을 선제적으로 수행하는 자동화 방안을 제시한다. 이를 통해 낮은 오버헤드로 관측 공백을 해소하고 안정적인 장애 회피가 가능함을 검증한다.

2. 이론적 배경

2.1 OpenStack on Kubernetes(OpenStack-Helm)

OpenStack on Kubernetes는 클라우드 인프라를 구축하는 오픈스택의 제어부(Control Plane) 및 주요 구성 요소를 쿠버네티스의 파드(Pod) 형태로 운영하는 아키텍처이다.

선언적 상태 관리 및 고가용성 보장 등 쿠버네티스의 핵심 기능을 활용함으로써, 오픈스택의 배포 및 서비스 관리 효율성을 향상시킬 수 있다. 이러한 배포 방식을 구현한 대표적인 프레임워크로 OpenStack-Helm(OSH)이 있다.

2.2 Libvirt 통계 기반 모니터링(virsh domstats)

libvirt가 제공하는 통계 수치를 통해 VM의 CPU 사용 시간, 네트워크 트래픽, 스토리지 입출력(I/O) 용량 및 속도 등의 지표를 수집할 수 있다. 오픈스택 실로미터(Ceilometer) 컴포넌트가 libvirt 기반 통계를 제공한다. 그러나 해당 통계는 단순 자원 사용량만을 나타내며, libvirt는 경합 상태 확인을 위해 vcpu.N.delay 필드를 제공하기는 하지만[3] 이는 단편적인 누적 대기 시간일 뿐 커널의 PSI(Pressure Stall Information) 규격을 지원하지 않는다. 따라서 외부 모니터링 및 자동화 파이프라인과 직접 연동하는 작업에 적합하지 않다.

2.3 쿠버네티스 표준 모니터링(kubelet/cAdvisor)

cAdvisor를 내장하는 kubelet는 노드 전체 차원의 자원 사용량은 계측할 수 있으나, cAdvisor는 관측 대상을 kubepods cgroup 계층으로 한정하여 호스트의 machine(.slice) cgroup은 탐색하지 않는다[4]. 이때문에 OpenStack-Helm 환경에서 워커 노드(Worker Node)의 QEMU 프로세스는 machine(.slice) cgroup 하위에

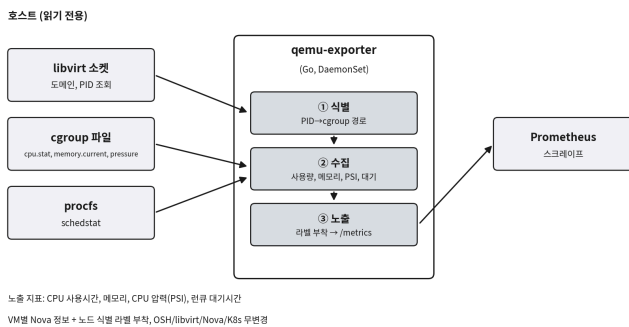
배치되므로, 노드 전체 자원 중 개별 VM이 점유하는 자원 사용량을 식별하여 측정하지 못하는 관측 공백이 발생한다. 또한 쿠버네티스의 스케줄링 및 파드 관리 메커니즘에서도 구조적 문제를 유발한다. QEMU 프로세스의 자원 점유는 쿠버네티스 스케줄러가 파드를 배치할 적절한 노드를 판단하는 자원 요청량(requests) 데이터에 반영되지 않으므로, 스케줄러는 위커 노드의 실제 자원이 고갈되어 감에도 이를 인지하지 못하고 파드를 추가 배치할 수 있다. 이는 결과적으로 자원 고갈 발생 시 VM 및 필수 컴포넌트 성능 저하를 유발하여 위커 노드의 인프라 기능 저하의 원인이 된다.

3. 연구 방법

3.1 qemu-exporter 모니터링 에이전트

첫 번째 목표로 본 연구는 OpenStack-Helm(OSH), libvirt, Nova 및 쿠버네티스 소스코드의 수정 없이, 호스트의 libvirt 소켓과 cgroup/procfs 파일 시스템을 읽어 외부 모니터링 서비스인 프로메테우스(Prometheus) 규격으로 노출하는 읽기 전용(Read-Only) 수집기 qemu-exporter를 제안한다. 에이전트의 주요 동작은 다음 세 단계로 구성된다.

<그림 1> qemu-exporter 구성도



첫째, “식별 단계”이다. libvirt 읽기 전용 소켓을 사용해서 동작 중인 VM의 PID를 알아내고, /proc/<PID>/cgroup 경로에서 도메인 스레드 탐색 알고리즘을 통해서 자원 계측을 위한 cgroup 경로를 찾는다. 둘째, “수집 단계”이다. 확보한 cgroup 경로에서 CPU 사용시간, 메모리 사용량(memory.current), CPU의 PSI 데이터(cpu.pressure)를 측정하고, procfs(schedstat)에서 스레드별 CPU 런큐 대기 시간을 측정한다. 셋째, “노출 단계”이다. 수집된 VM의 자원 사용량 및 자원 경합 지표에 Nova 메타데이터(인스턴스 ID, 이름, 인스턴스 타입 등)와 노드 식별 라벨을 결합하여 프로메테우스 서버가 스크레이프(Scrape)할 수 있는 표준 규격(/metrics)으로 노출한다.

본 수집기는 빠른 실행 속도와 호스트 컴퓨팅 자원의 오버헤드 최소화를 위해 Go 언어로 구현되었으며, 쿠버네티스 기반 인프라 환경과의 친화성을 위해 컨테이너 이미지로 빌드되어 파드(Pod) 형태로 배포된다.

3.2 실험 설계

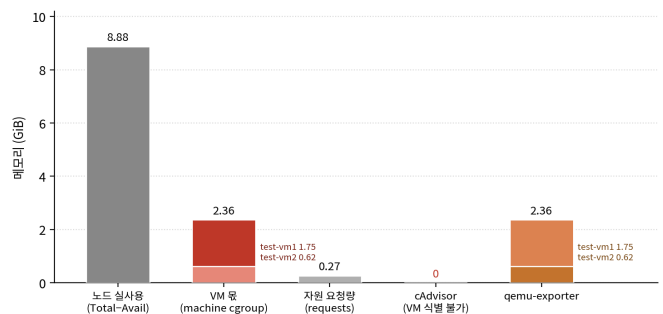
AWS EC2 클라우드 인스턴스 m5.2xlarge (8 vCPU, 32

GiB Memory) 환경에서 Ubuntu 24.04.4 LTS OS를 기반으로 실험을 구성하였다. OpenStack-Helm(OSH)을 활용하여 오픈스택의 제어부(Control Plane) 및 주요 구성 요소를 쿠버네티스 파드(Pod) 형태로 배포하였으며, 오픈스택을 통해 관측 대상 VM 2대를 생성한 후 실측 실험을 진행하였다.

3.3 실험 진행 및 결과

qemu-exporter의 필요성과 유효성을 검증하기 위해 세 가지 실험을 수행하였다. 첫 번째 실험에서는 VM이 실제로 점유하는 메모리가 쿠버네티스의 관리 수치 및 모니터링 체계에 어떻게 반영되는지 검증하였다.

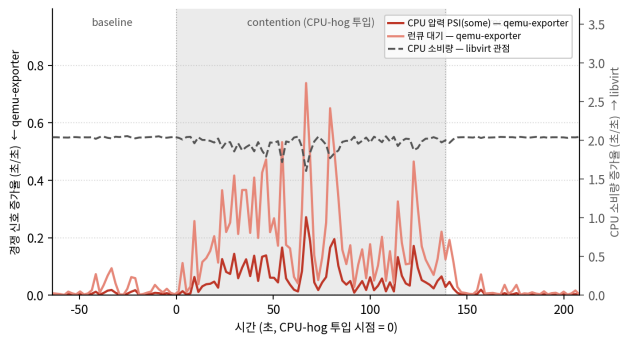
<그림 2> 메모리 사용량 인식 여부 : kubelet/cAdvisor와 자원 요청량, qemu-exporter



실험 환경에서 VM은 노드 전체 실사용 메모리의 27%(2.36 GiB)를 점유하도록 배포되었다. 그러나 자원 요청량(requests) 데이터는 272 MiB에 불과하였다. 이는 쿠버네티스의 스케줄링 관리 대상에 속하지 않는 QEMU 프로세스의 실제 자원 점유가 노드 배치 판단 과정에서 누락되어 있음을 보여준다. 또한 쿠버네티스의 표준 모니터링 도구(cAdvisor)는 자신이 관리하는 cgroup만 측정하므로, 노드 전체 메모리 수치에는 반영되는 VM의 실사용량(2.36 GiB)을 식별 및 분류하지 못하는 관측 공백을 드러냈다. 반면, qemu-exporter는 호스트 cgroup 파싱을 통해 각 VM의 메모리 사용량을 정확히 측정함과 동시에, 이를 Nova 메타데이터(인스턴스 ID, 이름, 인스턴스 타입 등)와 결합하여 정밀하게 노출함을 확인하였다.

두 번째 실험에서는 CPU 자원 경합 상황을 유발하고, 이때 기존 libvirt 모니터링 그리고 제안한 qemu-exporter가 각각 경합을 측정할 수 있는지 비교 분석하였다. 부하 인가 전 60초, CPU 부하 인가 120초, 부하 종료 후 60초로 구성된 시나리오에서 2초 간격으로 각 도구의 CPU 사용률 및 PSI 데이터 등을 수집하였다.

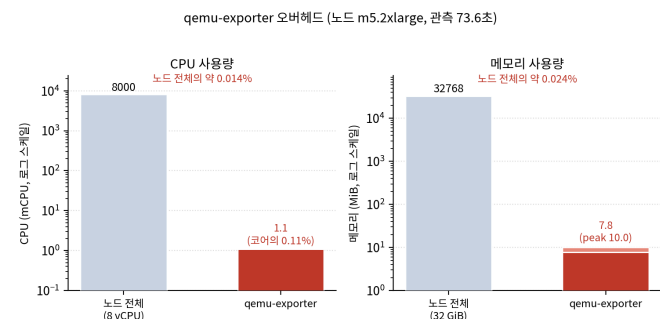
<그림 3> CPU 자원 경합의 노출 여부 : libvirt와 qemu-exporter



cgroup 내 스레드가 CPU를 할당받지 못한 시간 비율을 나타내는 PSI(some) 지표와 스레드별 대기 시간의 총합을 나타내는 schedstat(런큐 대기) 지표를 파악하는 qemu-exporter는 부하 발생 시 CPU 미할당 상태를 포착하였다. 구체적으로 부하 상황에서 PSI(some) 지표는 12.4배(0.59% => 7.3%), 런큐 대기 지표는 10.2배(0.045 => 0.463) 상승하였다. 반면, libvirt 모니터링은 CPU 사용률 집계 시 대기 상태를 반영하지 않아[3], CPU 미할당 상태에도 CPU 사용률이 3% 감소한 것으로만 측정되었다.

세 번째 실험에서는 qemu-exporter가 모니터링 에이전트로 동작할 때 호스트에 가하는 시스템 오버헤드를 측정하였다.

<그림 4> qemu-exporter가 호스트 컴퓨팅 자원을 차지하는 정도



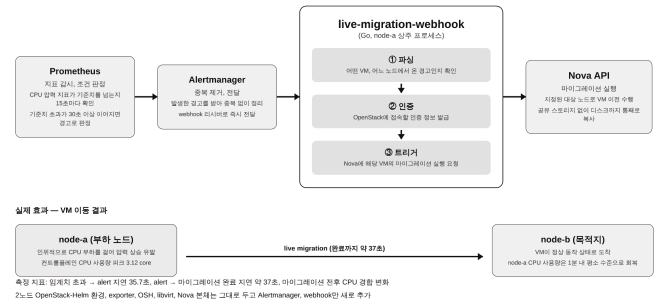
측정 결과 단일 코어 기준 0.11%, 노드 전체(8 vCPU) 기준으로는 약 0.014%에 불과했으며, 메모리 사용량은 피크 시에도 10 MiB 수준에 그쳐 호스트 컴퓨팅 자원에 미치는 영향이 미미함을 확인하였다.

3.4 qemu-exporter 기반 VM 라이브 마이그레이션

두 번째 목표로, qemu-exporter가 수집하는 CPU PSI 지표를 Prometheus, Alertmanager로 감지해 webhook을 통해 Nova live migration을 자동 실행하는 실험을 2노드 OpenStack-Helm 환경에서 수행했다.

<그림 5> qemu-exporter 기반 라이브 마이그레이션

선제적 Live Migration 파이프라인 (목표2)



PSI 임계치 초과 시 Alertmanager가 alert를 발생시키고 자체 구현한 webhook 리시버가 이를 받아 Nova API로 마이그레이션을 트리거하는 파이프라인을 구성했다. 대상 VM에 CPU 부하를 걸어 검증한 결과, 임계치 초과 후 alert 발생까지의 지연은 Prometheus 평가 주기 15초 기준 35.7초로 측정되었다. 또한 alert부터 마이그레이션 완료까지는 37~38초로 일정하게 측정되었다. 마이그레이션 후 부하가 걸렸던 노드의 컨트롤 플레인 CPU 사용량은 피크(코어 3.12) 대비 절반 이하로 1분 내 회복되어, 자원 고갈 전에 VM을 선제적으로 회피시킬 수 있음을 실증했다.

4. 결론

본 연구는 OpenStack-Helm(OSH) 환경에서 QEMU 프로세스가 호스트 machine(slice) cgroup에 배치되어 발생하는 구조적 관측 및 관리 공백을 실측 검증하였다. 검증 결과, 쿠버네티스 스케줄러의 자원 요청량 누락, cAdvisor의 VM 자원(메모리의 약 27%) 미식별, 자원 경합 시 기존 libvirt 지표의 감지 한계(PSI 및 런큐 대기 지표의 10~12배 증가에도 CPU 사용량은 3% 감소)를 확인하였다. 이를 해결하고자 호스트 커널 데이터와 Nova 메타데이터를 결합하는 경량 수집기인 qemu-exporter를 제안하였다. 제안한 수집기는 쿠버네티스 수정 없이 Alertmanager 및 Webhook 기반 파이프라인과 연동하여 자원 고갈 전 Nova 라이브 마이그레이션을 선제 수행한다. 이를 통해 노드 자원 경합 시 VM의 성능을 일정하게 유지하고 오픈스택 컴포넌트의 실행 안정성을 확보하였다. 단, AWS EC2 에뮬레이션 환경 제약으로 인한 CPU Steal 지표 수집 한계는 향후 베어메탈 KVM 환경에서 후속 검증할 예정이다.

참고문헌

- [1] Cloud Native Computing Foundation, "Kubernetes Established as the De Facto 'Operating System' for AI as Production Use Hits 82% in 2025 CNCF Annual Cloud Native Survey," CNCF, 2026. (<https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/>) (accessed Sep. 2026)
- [2] libvirt, "Control Groups Resource Management," libvirt.org, 2026. (<https://libvirt.org/cgroups.html>) (accessed Sep. 2026)

- [3] libvirt, "virsh(1) Manual — domstats," libvirt.org, 2026.
(<https://libvirt.org/manpages/virsh.html#domstats>)
(accessed Sep. 2026)
- [4] Kubernetes, "Tools for Monitoring Resources,"
Kubernetes Documentation, 2026.
(<https://kubernetes.io/docs/concepts/cluster-administration/resource-usage-monitoring/>) (accessed Sep. 2026)